



UNIVERSIDAD
CATÓLICA DE
TEMUCO

INGENIERÍA CIVIL
EN INFORMÁTICA
FACULTAD DE INGENIERÍA

INFORME ACADÉMICO

Análisis léxico del lenguaje Prolog

Diseño, implementación y validación de un analizador léxico para un subconjunto de Prolog

ASIGNATURA	Teoría de la Computación · INFO1148
ESTUDIANTES	Fernando Valdes · Juan Muñoz Veloso · Vicente Rivera
PROFESOR	Prof. M. Lévano
ENTREGA	17 de septiembre de 2026
REPOSITORIO	github.com/FernandoValdes01/Tarea01TC

Resumen ejecutivo

Este informe presenta el análisis, diseño, implementación y validación de un analizador léxico para un subconjunto de Prolog inspirado en la notación de ISO Prolog y SWI-Prolog. El trabajo define formalmente el catálogo de tokens con una expresión regular por categoría, modela las categorías principales con autómatas finitos, muestra la determinización por construcción de subconjuntos y la minimización por refinamiento de particiones sobre un subconjunto representativo, e implementa el lexer en Python con recorrido de izquierda a derecha, máxima coincidencia y prioridades explícitas. El lexer emite tipo, lexema original, línea y columna por token, administra una tabla de lexemas con deduplicación por par tipo y lexema, y reporta seis clases de error con fragmento y posición continuando el análisis. La validación usa una suite de 85 pruebas automatizadas con 26 casos válidos, 22 de prioridad, 16 inválidos y dos archivos completos, con resultado real de 85 aprobadas y cero fallos.

Palabras clave: análisis léxico, Prolog, expresiones regulares, autómatas finitos, tabla de lexemas.

1. Introducción

El análisis léxico es la primera fase de un compilador: transforma la cadena fuente en una secuencia ordenada de tokens con posición, separando las decisiones regulares (qué formas son válidas) de las decisiones sintácticas (en qué orden pueden aparecer). Un lexer correcto y bien diagnosticado simplifica el parser, produce errores comprensibles y deja evidencia reproducible de cada decisión.

Este informe documenta el subconjunto de Prolog reconocido, su especificación formal, los autómatas que lo modelan, la implementación en Python, la tabla de lexemas, el tratamiento de errores y el plan de pruebas con resultados reales. Cada afirmación remite a su respaldo: sección de la especificación, módulo del código, prueba automatizada o salida de un comando ejecutado.

2. Objetivos

El objetivo general es analizar, diseñar, implementar y validar un analizador léxico para un subconjunto definido de Prolog, aplicando alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos, con pruebas reproducibles y diagnósticos verificables.

Los objetivos específicos son: definir formalmente cada categoría con nombre de token, patrón, ejemplos válidos e inválidos y atributo asociado; construir una expresión regular por categoría resolviendo superposiciones con máxima coincidencia y prioridades; modelar las categorías principales con AFN o AFD y presentar un AFD integrado o una estrategia equivalente con estados de aceptación identificados; mostrar la determinización por construcción de subconjuntos y la minimización con justificación de equivalencias sobre un subconjunto representativo; implementar el lexer con posiciones 1-based, tabla de lexemas y seis clases de error recuperables; y validar con al menos 20 pruebas válidas, 8 inválidas, casos de prioridad y dos archivos completos, registrando el resultado real de la suite.

3. Alcance del analizador

El analizador reconoce exactamente el catálogo de docs/informe/01_especificacion_lexica.md: átomos simples y entre comillas simples, variables nombradas y anónima, enteros y reales sin signo, strings con comillas dobles, los operadores de las seis familias (cláusula, consulta, DCG, unificación y comparación, aritméticos, control), los nueve delimitadores de un carácter y los elementos ignorables (espacios, tabulaciones, saltos de línea y comentarios de línea y de bloque).

Quedan explícitamente fuera del alcance el análisis sintáctico y semántico: no se verifica el orden de los tokens, no se construye un árbol sintáctico, no se comprueba el balance de paréntesis, corchetes o llaves, no se resuelven ámbitos ni variables repetidas, no se implementa unificación, no se evalúan expresiones y no se decide si una secuencia de tokens forma un programa válido. Una `)` aislada es un token `RPAREN` correcto; su posible improcedencia pertenece a una fase posterior.

4. Especificación léxica

El contrato completo vive en docs/informe/01_especificacion_lexica.md y este capítulo lo resume sin modificarlo. El lexema de cada token es el segmento original de entrada, sin normalización ni conversión de números y sin des-escapar literales. El alfabeto de identificadores es ASCII y cualquier letra, dígito o espacio Unicode fuera de un literal es un carácter no admitido.

Token	Descripción	Patrón Python (<code>re</code>)	Ejemplos válidos	Exclusiones	Atributo
ATOM	Átomo simple iniciado por minúscula ASCII.	<code>[a-z][a-z0-9_]*</code>	padre, persona_1, isla	Padre, á	ninguno
QUOTED_ATOM	Átomo delimitado por comillas simples.	<code>'(?:[^\\"\\r\\n] \\\\"\\nrt \\'*)'</code>	'Juan Pérez', ':-', 'it's'	'sin cierre, 'a\\q', \\'' interno	ninguno
VARIABLE	Identificador iniciado por mayúscula o <code>_</code> con continuación.	<code>(?:[A-Z][A-Za-z0-9_]* _[A-Za-z0-9_]+)</code>	X, Persona, _Temporal, _1	_solo, 9X	ninguno
ANONYMOUS_VARIABLE	Guion bajo aislado.	<code>_</code>	<code>_</code>	<code>_X</code> , <code>__</code> (son VARIABLE)	ninguno
INTEGER	Secuencia de dígitos ASCII sin signo.	<code>[0-9]+</code>	0, 25, 0007	-25, 5., .5	ninguno
REAL	Dígitos, punto y dígitos, sin signo.	<code>[0-9]+\\. [0-9]+</code>	3.14, 0.5, 10.00	5., .5, 1.2.3, exponentes	ninguno
STRING	Literal delimitado por comillas dobles.	<code>"(?:[^\\"\\r\\n] \\\\"\\nrt \\'*)"</code>	"hola", "dice \\\"si\\\"", "línea\\n"	"sin cierre, "a\\q", \\'" interno	ninguno
OPERATOR	Operador exacto de una de seis familias.	unión normativa de la sección 6	<code>:-</code> , <code>?-</code> , <code>--></code> , <code>\\==</code> , <code>=..</code> , <code>//</code> , <code>**</code> , <code>is</code> , <code>mod</code> , <code>\\+</code> , <code>!</code> , <code>;</code>	<code>!?</code> y una barra inversa aislada	family
LPAREN ... DOT	Delimitadores de un carácter.	una alternativa por símbolo	<code>(</code> , <code>)</code> , <code>[</code> , <code>]</code> , <code>{</code> , <code>}</code> , <code>\\ </code> , <code>,</code> , <code>.</code>	sin validación de balance	ninguno

Las familias de OPERATOR son CLAUSE (`:`, `-`), QUERY (`?`, `-`), DCG (`-->`), UNIFICATION_COMPARISON (`=`, `\\=`, `==`, `\\==`, `=..`, `<`, `=<`, `>`, `>=`), ARITHMETIC (`+`, `-`, `*`, `/`, `//`, `**`, `is`, `mod`) y CONTROL (`\\+`, `!`, `;`). La unión normativa se implementa como un mapa de operadores recorrido por longitud descendente. Las palabras `is` y `mod` se reconocen solo cuando el lexema completo no tiene continuación de identificador.

Las convenciones adoptadas son: los números no tienen signo (`-12` son dos tokens); INTEGER y REAL no admiten exponentes ni puntos sin dígitos a ambos lados; los escapes válidos son exactamente `\\`, `\\'` o `\\"` según el delimitador, más `\\n`, `\\r` y `\\t`, y cualquier otra pareja es INVALID_ESCAPE; ningún literal admite un salto de línea real en su interior; `%` inicia el comentario de línea hasta antes del salto; `/*` abre un bloque que termina en el primer `*/` sin anidamiento; y las posiciones son 1-based con tabulación de una columna y `\\r\\n` contado como un único salto.

5. Diseño de autómatas

Las figuras usan la convención estado `--símbolo-->` estado; un estado cuya etiqueta contiene `/` TOKEN es aceptor y declara el token reconocido. Los alfabetos abreviados son LOWER = `[a-z]`, UPPER = `[A-Z]`, DIGIT = `[0-9]` e ID = `[A-Za-z0-9_]`.

Figura 1, AFD de átomos: `q0 --LOWER--> ((q1))` y `q1 --[a-z0-9_]-> q1`. El estado inicial es `q0` y el único estado de aceptación es `q1`, que reconoce ATOM. Cualquier otro primer carácter no tiene transición y el dispatcher del lexer lo deriva a otra categoría o a error.

Figura 2, AFD de variables y anónima: `q0 --UPPER--> ((q1))`, `q0 --_--> q2`, `q1 --ID--> q1`, `q2 --ID--> ((q3))` y `q3 --ID--> q3`. El estado `q1` reconoce VARIABLE mayúscula, `q3` reconoce VARIABLE con guion bajo y el estado `q2` sin continuación se resuelve por la regla prioritaria de `_` aislado como ANONYMOUS_VARIABLE, nunca como variable.

Figura 3, AFD de números: `n0 --DIGIT--> ((nI INTEGER))`, `nI --DIGIT--> nI`, `nI --'.--> nP`, `nP --DIGIT--> ((nR REAL))` y `nR --DIGIT--> nR`. Los únicos estados de aceptación son `nI` y `nR`; `nP` no

acepta, por lo que 5. no pertenece al lenguaje numérico. . 5 y un segundo punto tampoco tienen recorrido aceptador. El conductor del lexer detecta esas corridas rechazadas y las informa como MALFORMED_NUMBER; esta recuperación diagnóstica no convierte estados de error en estados aceptores.

Figura 4, trie de operadores simbólicos con prefijos comunes: desde s_0 , las ramas $\backslash$ llevan a $\backslash=$, $\backslash==$ y $\backslash+$; las ramas $=$ llevan a $=$, $==$, $=.$ y $=<$; $/$ lleva a $/$, $//$ y al desvío de comentario $/^*$; $*$ lleva a $*$ y $**$; $:$ solo continúa a $:-$; $?$ solo continúa a $?-$; $-$ continúa a $-->$; $<$, $>$, $+$, $!$, $;$ completan las ramas de uno o dos caracteres. Cada nodo de aceptación porta su lexema y su familia; los nodos intermedios sin aceptación (por ejemplo tras $\backslash$ o $:$) no emiten token si la entrada termina allí.

Figura 5, AFD de ignorables: espacios y tabulaciones ciclan en consumo de una columna; $\backslash n$, $\backslash r$ y $\backslash r\backslash n$ producen un salto lógico con reinicio de columna; $\%$ consume hasta antes del salto; y $/*$ entra al cuerpo del bloque, que cicla hasta el primer $*/$ o cae al error UNTERMINATED_BLOCK_COMMENT al llegar a EOF.

Figura 6, autómata integrado o estrategia equivalente: el lexer implementa un despachador por primera clase de carácter (dígito o punto con dígito siguiente hacia números; comilla hacia literales; letra o guion bajo hacia palabras; símbolo hacia el trie de operadores; espacios, $\%$ y $/*$ hacia ignorables) y en cada posición aplica máxima coincidencia con las prioridades de la sección 6. Esta estrategia es equivalente a unir por transiciones ϵ los autómatas anteriores, determinar el conjunto y etiquetar cada aceptación con su token y prioridad.

5.1 Determinización y minimización

Para mostrar el procedimiento completo se tomó el lenguaje representativo $\{=, ==, \backslash=, \backslash==\}$. El AFN de la Figura 7 une mediante ϵ dos ramas: $q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{=} q_2 \xrightarrow{=} q_3$ y $q_0 \xrightarrow{\epsilon} q_4 \xrightarrow{\backslash} q_5 \xrightarrow{=} q_6 \xrightarrow{=} q_7$. Los estados q_2 , q_3 , q_6 y q_7 aceptan respectivamente OP_EQ, OP_EQEQ, OP_NEQ y OP_NEQEQ. Se conservan etiquetas distintas porque el lexer debe producir lexemas distintos aunque todos pertenezcan a la familia UNIFICATION_COMPARISON.

La construcción de subconjuntos comienza con $A = \epsilon\text{-cerradura}(\{q_0\}) = \{q_0, q_1, q_4\}$. Después de esta cerradura inicial no existen nuevas transiciones ϵ , por lo que las cerraduras posteriores coinciden con el resultado de move. La Figura 8 y la tabla muestran todos los subconjuntos alcanzables, incluido el estado muerto Z:

Estado AFD	Subconjunto AFN	Con =	Con \	Aceptación
A	$\{q_0, q_1, q_4\}$	B	D	ninguna
B	$\{q_2\}$	C	Z	OP_EQ
C	$\{q_3\}$	Z	Z	OP_EQEQ
D	$\{q_5\}$	E	Z	ninguna
E	$\{q_6\}$	F	Z	OP_NEQ
F	$\{q_7\}$	Z	Z	OP_NEQEQ
Z	$\emptyset$	Z	Z	ninguna

Para minimizar el AFD como clasificador de tokens, la partición inicial separa no aceptores y cada salida observable: $P_0 = \{\{A, D, Z\}, \{B\}, \{C\}, \{E\}, \{F\}\}$. Las firmas de A, D y Z respecto de P_0 son distintas: A conduce a $\{B\}$ y $\{A, D, Z\}$, D conduce a $\{E\}$ y $\{A, D, Z\}$, y Z permanece en $\{A, D, Z\}$ con ambos símbolos. Por ello el bloque se refina a $\{A\}$, $\{D\}$ y $\{Z\}$. La partición estable $P_1 = \{\{A\}, \{D\}, \{Z\}, \{B\}, \{C\}, \{E\}, \{F\}\}$ no permite más fusiones; el AFD mínimo etiquetado de la Figura 9 conserva siete estados y las cuatro salidas. Si se ignoraran las etiquetas podrían fusionarse algunos aceptores, pero se perdería la clasificación léxica requerida.

6. Reglas de prioridad y máxima coincidencia

En cada posición se elige el candidato válido más largo y solo en empate se aplica la prioridad contractual. La tabla verifica cada regla contra su prueba; family abrevia el atributo del operador.

Entrada	Candidatos	Regla ganadora	Resultado	Prueba
$\backslash==$	$\backslash==, \backslash=$	más largo	OPERATOR unificación	P01

Entrada	Candidatos	Regla ganadora	Resultado	Prueba
= . .	= . . , =	más largo	OPERATOR unificación	P02
==	==, =	más largo	OPERATOR unificación	P03
\=	\=	único	OPERATOR unificación	P04
: -	: - frente a : inválido	operador de dos caracteres	OPERATOR cláusula	P05
? -	? - frente a ? inválido	operador de dos caracteres	OPERATOR consulta	P06
-->	-->, -	más largo	OPERATOR DCG	P07
//	//, /	más largo	OPERATOR aritmético	P08
**	**, *	más largo	OPERATOR aritmético	P09
_	anónima frente a variable no aplicable	regla del anónimo	ANONYMOUS_VARIABLE	P10
_Tmp	variable completa frente a _	más largo	VARIABLE	P11
isla	ATOM(4) frente a prefijo is	más largo y guarda	ATOM	P12
is	ATOM y OPERATOR de igual longitud	palabra reservada	OPERATOR aritmético	P13
mod	ATOM y OPERATOR de igual longitud	palabra reservada	OPERATOR aritmético	P14
modulo	ATOM(6) frente a prefijo mod	más largo y guarda	ATOM	P15
1.25	REAL frente a INTEGER('1')	más largo, REAL antes que INTEGER	REAL	P16
X=Y	tres tokens adyacentes	el cursor avanza sin separadores	X, =, Y	P17
X\==Y	operador largo adyacente	máxima coincidencia	X, \==, Y	P18
is(	is más (no continuador	guarda de palabra	OPERATOR + LPAREN	P19
a: - b	cláusula adyacente	operador de dos caracteres	a, : -, b	P20
12abc	candidato numérico con sufijo inválido	recuperación hasta separador seguro	MALFORMED_NUMBER, sin tokens	P21
is_	ATOM(3) frente a prefijo is	guarda (_ es continuador)	ATOM	P22
5., .5, 1..2	candidato numérico inválido	detector antes que DOT	MALFORMED_NUMBER sin DOT	I07–I09

El caso 12abc aplica la recuperación contractual: se consume como un único fragmento numérico inválido y no se emiten tokens parciales que aparenten validez.

7. Diseño e implementación

7.1 Arquitectura de la solución

La arquitectura tiene cinco módulos con una sola dirección de dependencias: `tokens.py` define `TokenType`, `OperatorFamily`, el mapa único `OPERATOR_FAMILY` y el `Token` con posición de inicio y atributos opcionales; `errors.py` define `LexicalErrorKind` y `LexicalError`; `symbol_table.py` implementa `SymbolTable` con deduplicación por par tipo y lexema; `lexer.py` expone la clase `Lexer`, `tokenize(source)` y `tokenize_file(path)`; y `main.py` ofrece la CLI.

El flujo de datos recorre la entrada de izquierda a derecha con un cursor de índice, línea y columna: primero consume ignorables (espacios, saltos y comentarios), luego procesa literales, números, identificadores, operadores por longitud descendente, delimitadores y finalmente `INVALID_CHARACTER`. Cada token guarda la línea y columna de su primer carácter; la tabulación avanza una columna y `\r\n` cuenta como un único salto.

7.2 Formato de salida

La salida básica conserva <TIPO, 'lexema', línea, columna>; --show-attributes agrega la familia o el índice y --tab la vuelca la tabla. Un fragmento real del programa válido es:

Código (text)

```
<ATOM, 'padre', 2, 1>
<LPAREN, '(', 2, 6>
<ATOM, 'juan', 2, 7>
<COMMA, ',', 2, 11>
<ATOM, 'ana', 2, 13>
<RPAREN, ')', 2, 16>
<DOT, '.', 2, 17>
```

El archivo completo produce 207 tokens, 0 errores y 48 entradas de tabla. El programa con errores produce los 12 diagnósticos de la sección 8 y 69 tokens recuperados. Los comandos reproducibles son `python -m compileall src tests,uv run --with pytest python -m pytest -q,python -m src.main tests/corpus/entradas/programa_valido.py python -m src.main tests/corpus/entradas/programa_con_errores.pl`.

7.3 Tabla de lexemas

Se registran ATOM, QUOTED_ATOM y VARIABLE como identificadores, más INTEGER, REAL y STRING como literales conservables, siempre con el lexema original y sin conversiones. La clave de deduplicación es el par tipo y lexema, de modo que dos apariciones idénticas comparten índice y dos formas textuales distintas nunca se fusionan. Cada token registrable porta su `table_index` puramente léxico.

La variable anónima `_` nunca se registra porque cada aparición es independiente y no un identificador reutilizable. Operadores, delimitadores, espacios y comentarios tampoco se registran. La tabla no guarda alcances, posiciones de uso, unificaciones ni tipos semánticos; la prueba T08 verifica la deduplicación comparando conjuntos ordenados para no depender del orden interno del diccionario y T09 verifica que lo no registrable deja la tabla vacía.

8. Tratamiento de errores léxicos

Los seis errores contractuales se acumulan sin detener el recorrido y cada diagnóstico incluye clase, fragmento, línea, columna y mensaje útil. El átomo sin cierre descarta hasta antes del salto o hasta EOF (I01, I02); el string sin cierre hace lo mismo con su delimitador (I03, I04); el bloque `/* sin */` consume hasta EOF actualizando posiciones y no admite tokens posteriores (I05); el carácter no admitido (`@`, `:`, `?`, `\` aislada, `á` y solo espacio, tabulación y saltos como ignorables) consume un carácter y continúa (I06, I13–I16); el número mal formado consume la corrida de dígitos y puntos, más un posible sufijo contiguo de identificador, y nunca emite tokens parciales para ese fragmento (I07–I10, P21); y cada escape desconocido registra un `INVALID_ESCAPE` en la barra inversa, consume la pareja y sigue buscando el cierre, de modo que un literal con escapes inválidos no emite token y uno además sin cierre suma el error de cierre (I11, I12).

Error	Ejemplo	Mensaje producido	Recuperación
Átomo sin cierre	' Juan	UNTERMINATED_QUOTED_ATOM, ubicación y fragmento	descartar hasta salto o EOF
Cadena sin cierre	" Juan	UNTERMINATED_STRING, ubicación y fragmento	descartar hasta salto o EOF
Comentario sin cierre	/* texto	UNTERMINATED_BLOCK_COMMENT	consumir hasta EOF
Carácter no admitido	@	INVALID_CHARACTER('@')	consumir un carácter y continuar
Número mal formado	1..2	MALFORMED_NUMBER('1..2')	consumir la corrida completa
Escape inválido	'a\q'	INVALID_ESCAPE('\q')	consumir la pareja y buscar cierre

La salida real del archivo con errores contiene exactamente estos 12 diagnósticos:

UNTERMINATED_QUOTED_ATOM en 3:6, UNTERMINATED_STRING en 5:6, INVALID_CHARACTER('@') en 7:6, MALFORMED_NUMBER('5.') en 8:6, MALFORMED_NUMBER('.5') en 9:6, MALFORMED_NUMBER('1..2') en 10:6, INVALID_CHARACTER(':') en 11:5, INVALID_CHARACTER('?') en 12:5,

INVALID_CHARACTER('\ ') en 13:5, INVALID_ESCAPE('\q') en 14:8, INVALID_ESCAPE('\q') en 15:8 y UNTERMINATED_BLOCK_COMMENT en 17:1 hasta EOF.

9. Plan de pruebas y resultados

La suite `tests/test_lexer.py` suma 85 pruebas: 3 del modelo Token, 26 válidas V01–V26 con tokens y posiciones exactas, 22 de prioridad P01–P22, 16 inválidas I01–I16 con clase, fragmento, posición y recuperación, 12 transversales T01–T12 de posiciones, comentarios, escapes, tabla y acumulación, y 6 de archivos F01–F06 incluyendo lectura de los dos programas completos, de los 8 archivos de `validos/` y de los 6 de `invalidos/`, más la verificación del mínimo de corpus. La matriz completa con el resultado real de cada caso vive en `docs/informe/04_matriz_de_pruebas.md`.

ID	Entrada o archivo	Objetivo	Resultado esperado	Resultado obtenido	Estado
V23	<code>padre(juan, ana).</code>	Hecho y posiciones	7 tokens, sin errores	7 tokens en columnas exactas	OK
P01	<code>\==</code>	Máxima coincidencia	Un OPERATOR('\==')	Un operador de unificación	OK
P21	<code>12abc</code>	Recuperación numérica	Un error, sin tokens parciales	MALFORMED_NUMBER 1:1	OK
I02	átomo sin cierre + línea válida	Recuperación tras salto	Error 1:1 y tokens en línea 2	Coincide con lo esperado	OK
F01	<code>programa_valido.pl</code>	Cobertura integral	0 errores, 17 tipos y 6 familias	207 tokens, 0 errores	OK
F02	<code>programa_con_error.res.pl</code>	Diagnósticos y continuidad	≥8 errores recuperables	69 tokens y 12 errores	OK

El archivo válido produce 207 tokens, 0 errores y 48 entradas, cubriendo los 17 tipos de token y las 6 familias. El archivo con errores produce 69 tokens, 12 errores de las 6 clases en 12 líneas distintas y 28 entradas, con tokens válidos después de cada error recuperable y ningún token después del bloque sin cierre final. El comando ejecutado fue `uv run --with pytest python -m pytest -q` con resultado real `85 passed`; además `python -m compileall src tests` terminó sin errores y `git diff --check` salió limpio.

9.1 Análisis de resultados

La cobertura es total sobre el catálogo: cada tipo de token y cada familia aparece en el corpus válido y en el archivo completo. Los casos límite incluyen cierres a EOF y antes de salto, escapes en ambos delimitadores, cinco formas numéricas mal formadas, caracteres aislados, CRLF, tabulación, comentarios dentro de literales, `-12` como dos tokens y `)` aislado como token.

La principal dificultad fue distinguir el punto de fin de cláusula del punto decimal: una cláusula que termina en número sin espacio (`mod 5.`) es MALFORMED_NUMBER, de modo que los archivos válidos escriben `5 ..`. La recuperación de `12abc` consume el fragmento completo conforme al contrato. El archivo de errores demuestra 69 tokens válidos alrededor de 11 errores recuperables y un último error de bloque que no deja tokens posteriores.

10. Conclusiones

El trabajo muestra que un lexer riguroso nace de decisiones contractuales explícitas antes que del código: alfabeto ASCII, números sin signo, escapes cerrados por delimitador, comentarios no anidables y máxima coincidencia con prioridades resolvieron todas las ambigüedades sin casos especiales en la implementación. Los objetivos se cumplieron de forma medible con 85 pruebas aprobadas, cobertura total de categorías y familias, y dos archivos completos que ejercitan el recorrido real.

Las limitaciones reales son que el lexer solo reconoce el subconjunto contratado y que los archivos válidos deben separar con espacio un número del punto final por la regla de número mal formado. Como mejoras futuras se proponen modos de salida adicionales (JSON o CSV de tokens), un visualizador de autómatas generado desde las expresiones del código y pruebas de rendimiento sobre archivos grandes, sin ampliar el subconjunto ni

entrar en análisis sintáctico.

11. Referencias

Las fuentes realmente consultadas se presentan con un formato académico consistente:

- Lévano, M. (2026). *Tarea: análisis léxico del lenguaje Prolog. Investigación, análisis, diseño, implementación y validación* (INFO1148, sem2_2026). Enunciado del curso.
- ISO/IEC. (1995). *ISO/IEC 13211-1:1995. Information technology—Programming languages—Prolog—Part 1: General core*.
- Python Software Foundation. (2026). *Regular expression operations (re)*. <https://docs.python.org/3/library/re.html>
- pytest-dev team. (2026). *pytest documentation*. <https://docs.pytest.org/>
- Astral Software. (2026). *uv documentation*. <https://docs.astral.sh/uv/>
- Equipo de la asignatura INFO1148. (2026). *Formato Informe Tarea*. Documento de pauta disponible en `../teoria-computacion/tareas/Formato Informe Tarea.pdf`.

La trazabilidad interna se completa con `docs/informe/01_especificacion_lexica.md`, `docs/automatas/`, `docs/informe/04_matriz_de_pruebas.md`, el código de `src/` y el corpus de `tests/corpus/`.

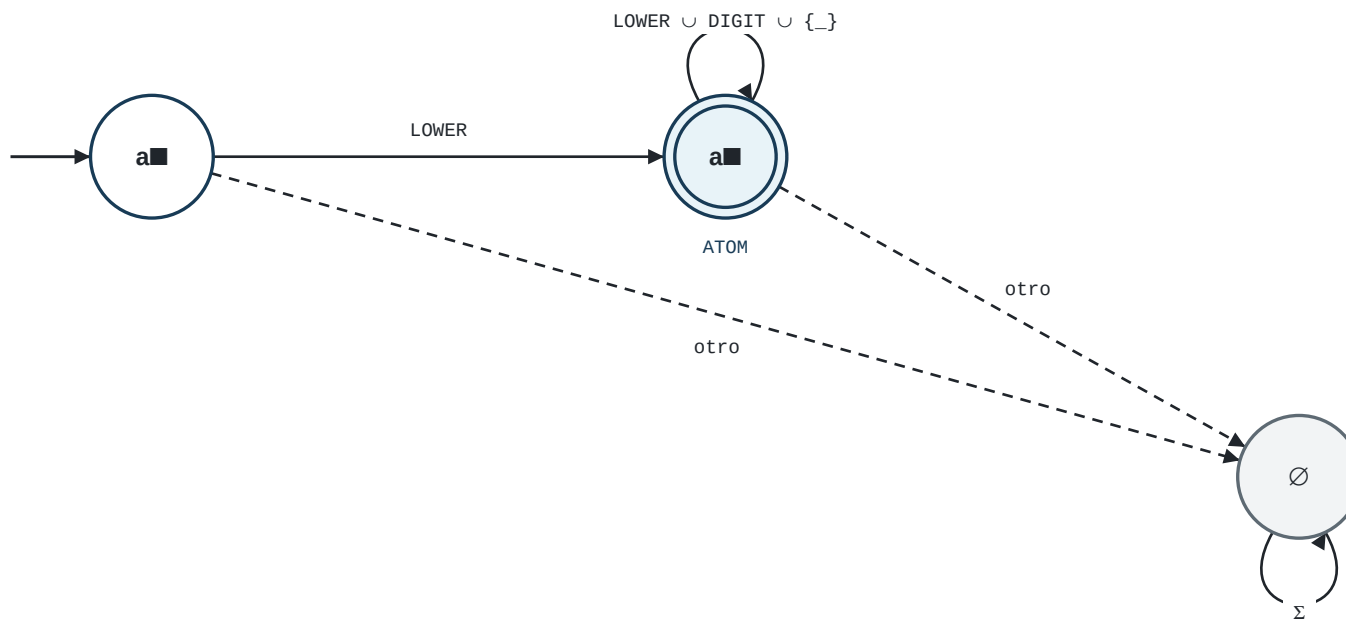
12. Anexos

Repositorio Git: <https://github.com/FernandoValdes01/Tarea01TC>

Los anexos documentales viven en `docs/informe/anexos.md`. El presente PDF incorpora nueve figuras vectoriales compuestas a partir de la especificación de estados y transiciones conservada en `docs/automatas/diagramas/`, incluida la secuencia coherente AFN → AFD por subconjuntos → AFD mínimo etiquetado.

AFD para átomos no entrecomillados

Reconoce $\text{LOWER} (\text{LOWER} \cup \text{DIGIT} \cup \{_ \})^*$ mediante máxima coincidencia.



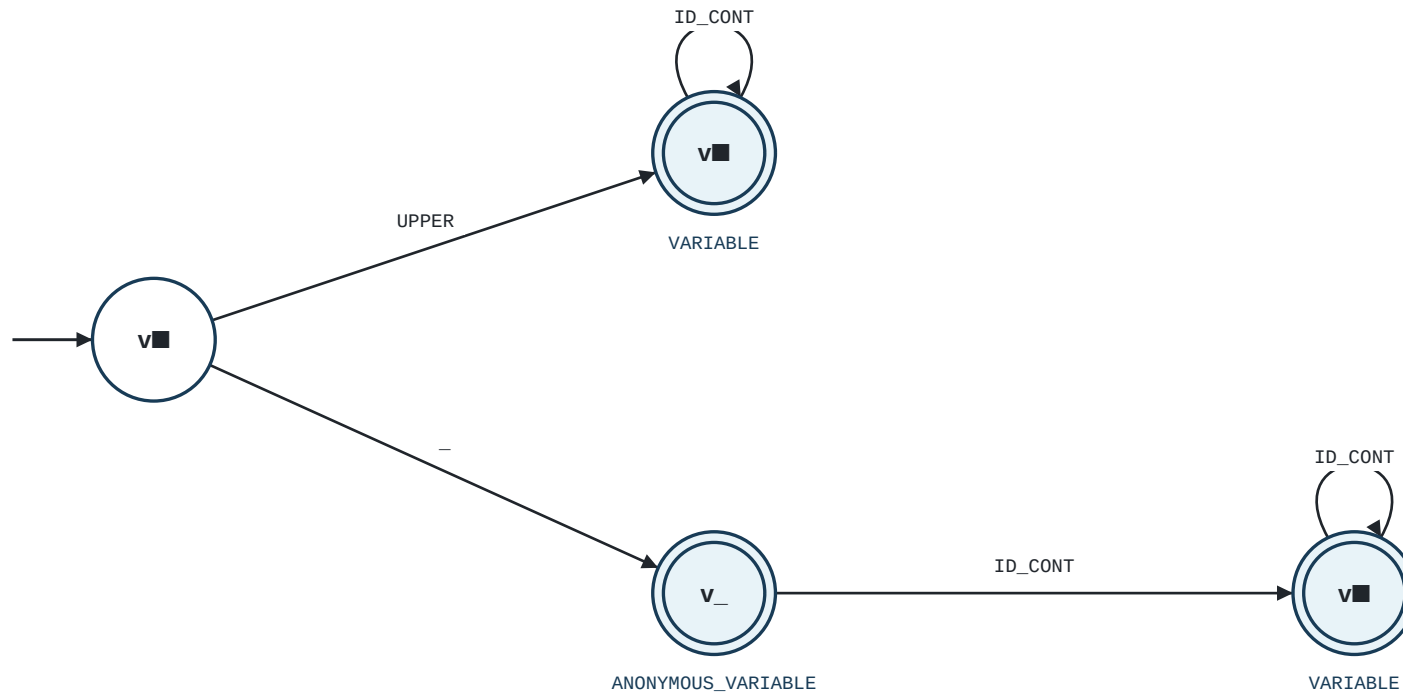
Estado aceptador y token emitido



Estado sumidero

AFD para variables

Distingue variables con nombre de la variable anónima ` \_`.



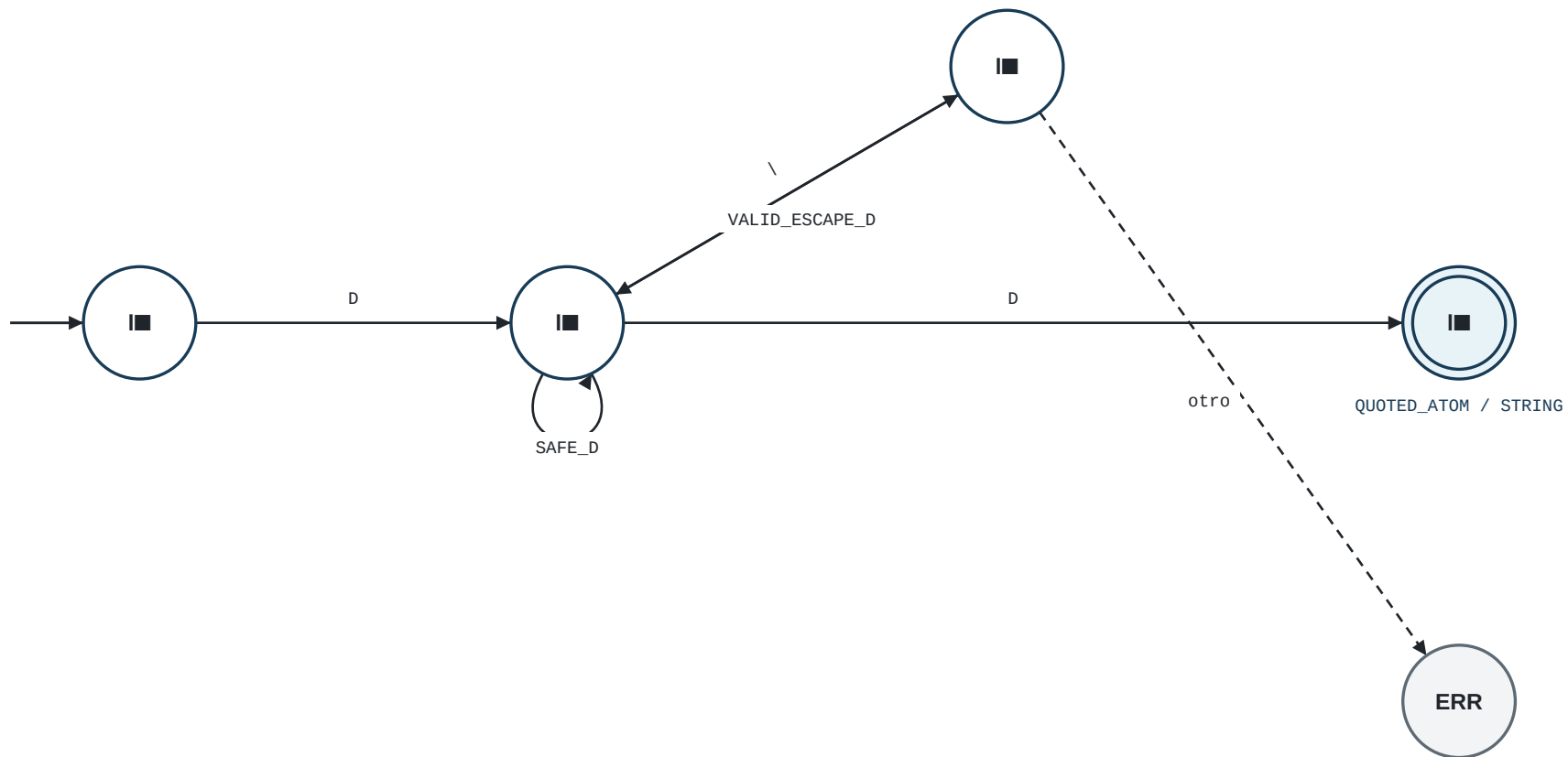
AFD para números

Reconoce enteros y reales sin signo; el punto decimal exige un dígito posterior.



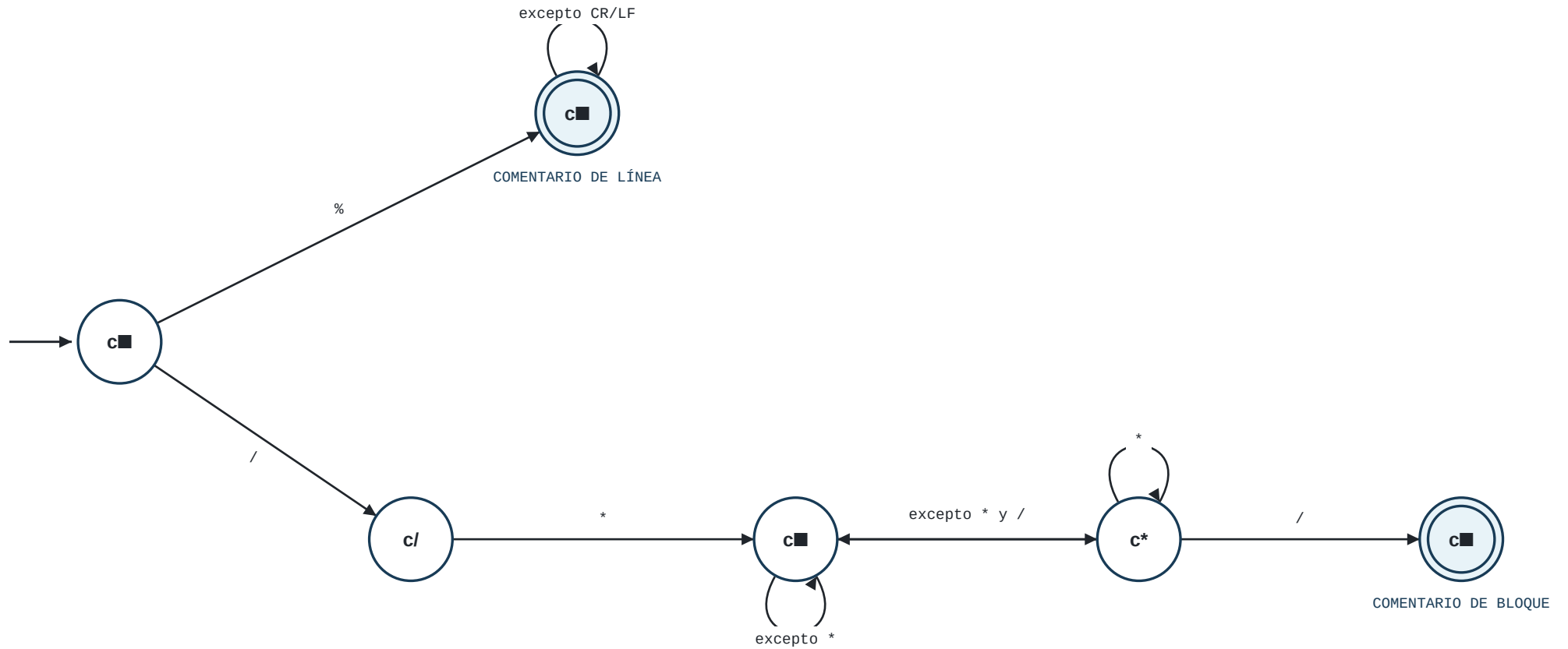
AFD para literales entrecomillados

Esquema parametrizado por delimitador D: comilla simple o comilla doble.



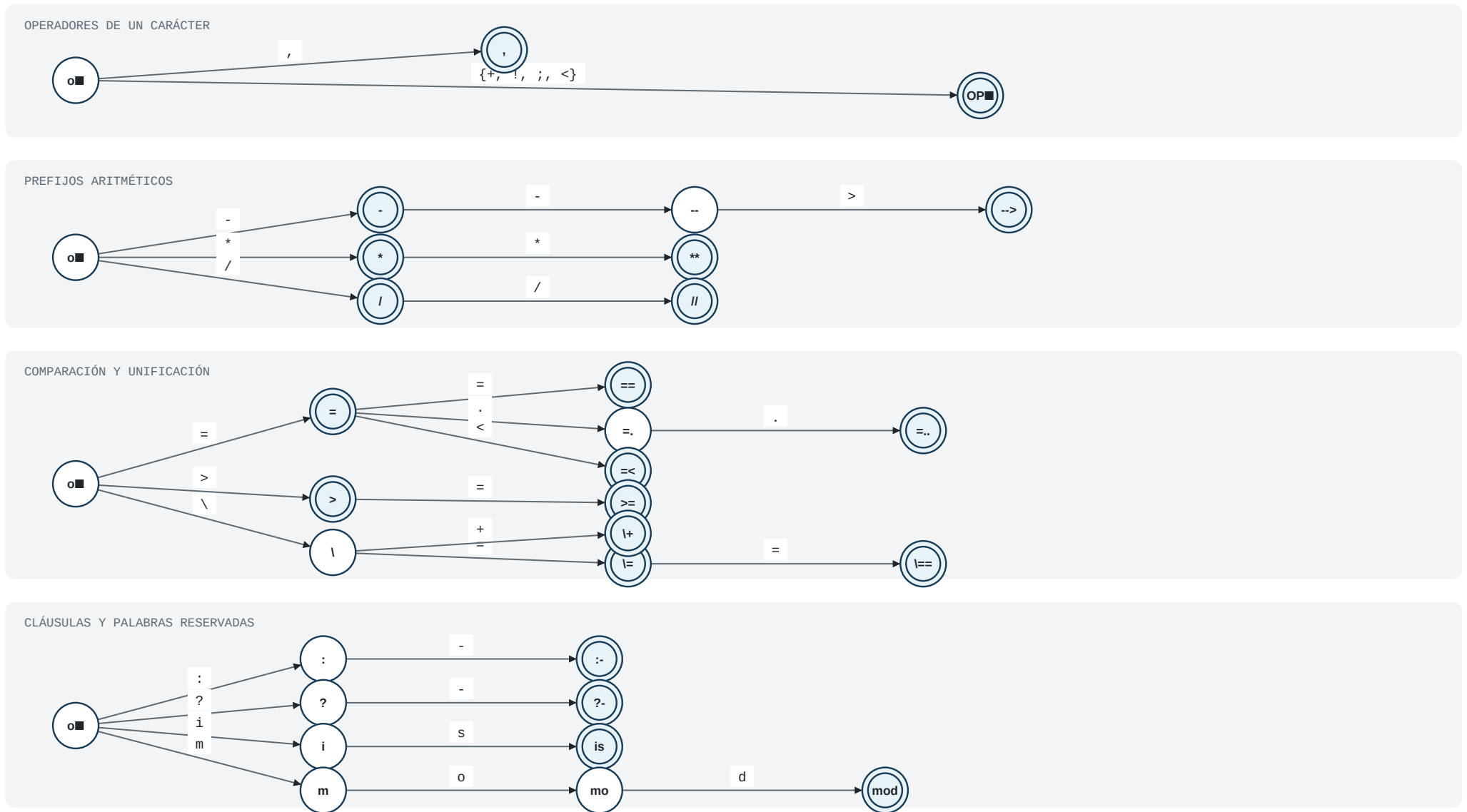
AFD para comentarios

Ramas independientes para comentario de línea y comentario de bloque.



Trie de operadores

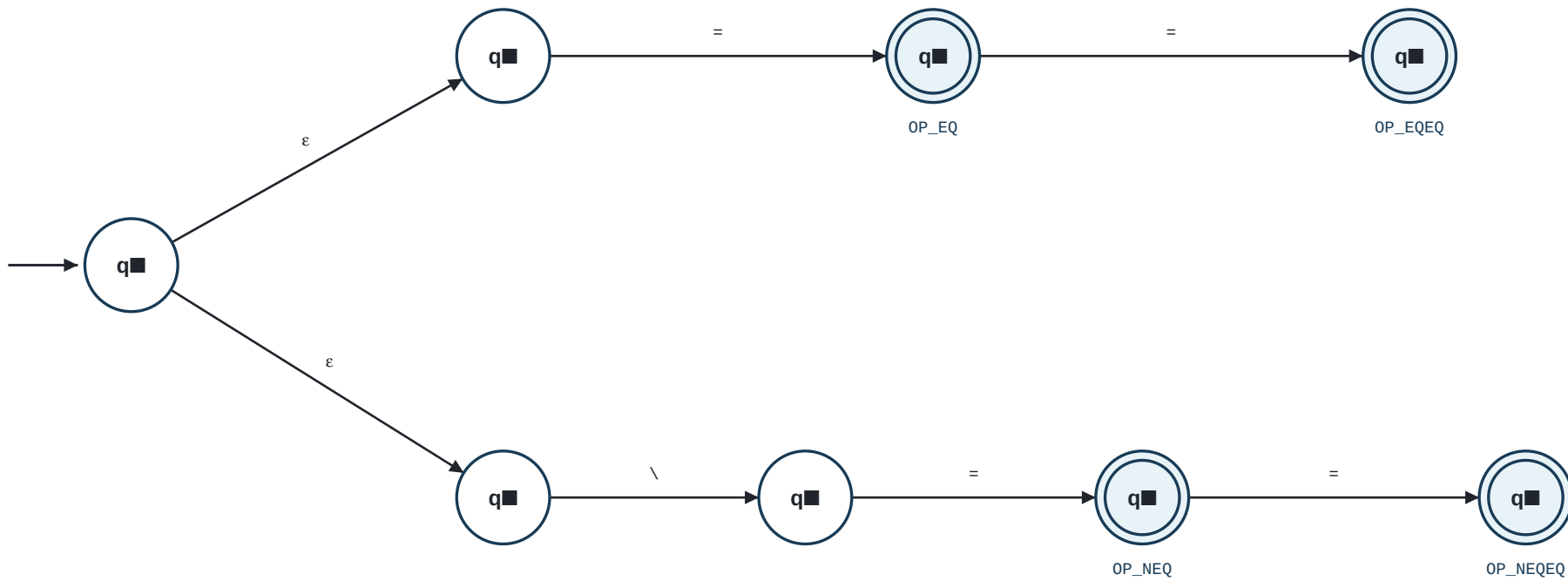
Organización por prefijo para aplicar máxima coincidencia antes de emitir el token.



`is` y `mod` requieren frontera de palabra.

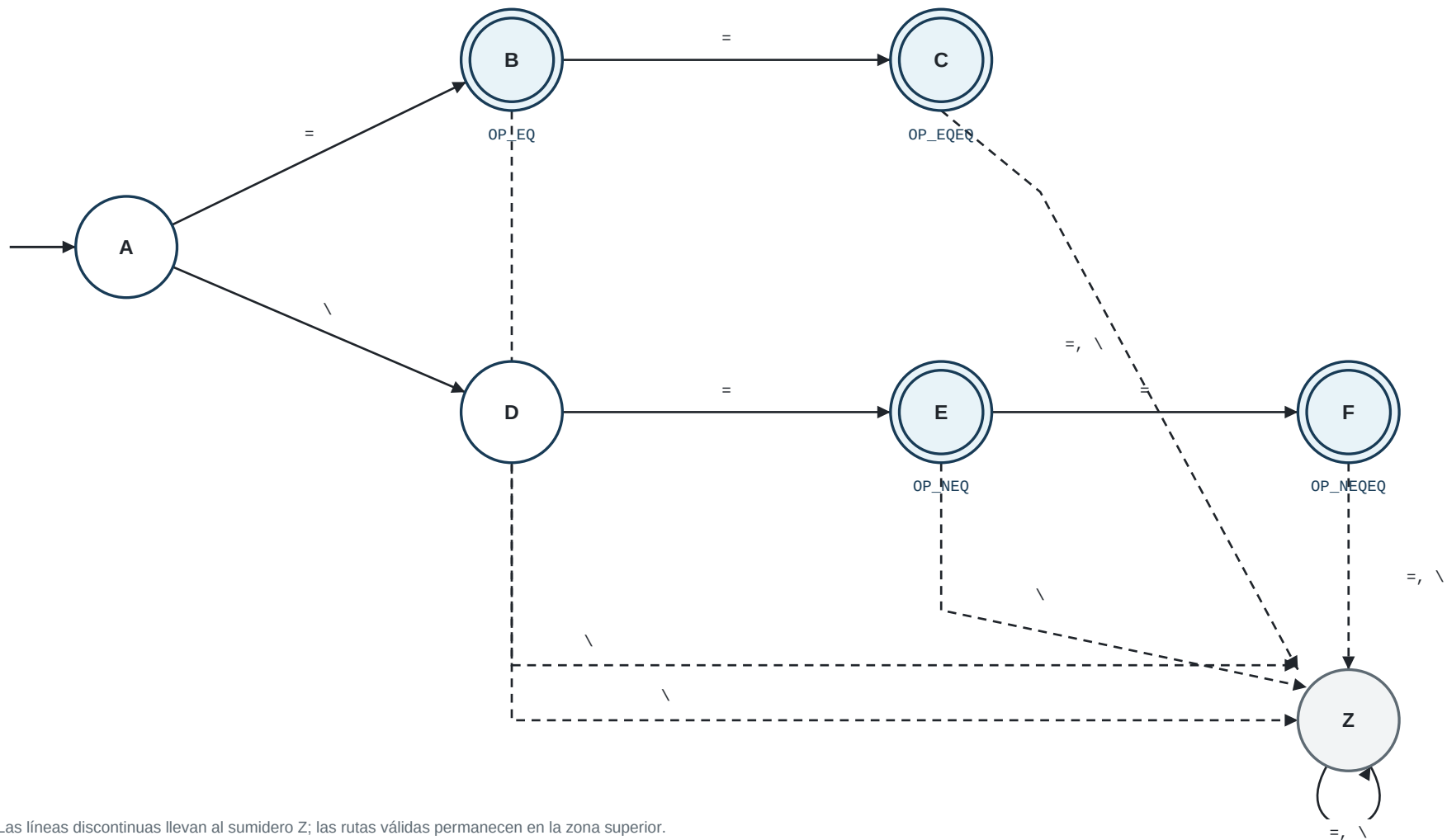
AFN representativo de operadores

Lenguaje restringido a $\{=, ==, \backslash=, \backslash==\}$ para mostrar la construcción formal.



AFD obtenido por construcción de subconjuntos

Cada estado representa un conjunto de estados del AFN; las salidas conservan la clasificación léxica.



AFD mínimo etiquetado

La partición estable separa A, D y Z y conserva cuatro clases de aceptación observables.

